

Σύστημα Συστάσεων eNOVATION CBRN

1. Εισαγωγή

Το παρόν έγγραφο αναλύει την αρχιτεκτονική και τη λογική του **eNOVATION CBRN Recommender**, ενός συστήματος λήψης αποφάσεων βασισμένου σε σημασιολογικούς κανόνες (semantic rule-based system). Σκοπός του συστήματος είναι η υπόδειξη του βέλτιστου Κέντρου Εκπαίδευσης (Training Centre - TC) βάσει δύο παραμέτρων εισόδου:

1. **Technology:** Η απαιτούμενη τεχνολογία.
2. **Scenario:** Το επιχειρησιακό σενάριο.

Το σύστημα χρησιμοποιεί έναν Γράφο Γνώσης (Knowledge Graph) για την εξαγωγή συμπερασμάτων και έναν Αλγόριθμο Βαθμολόγησης Πολλαπλών Κριτηρίων για την κατάταξη των αποτελεσμάτων.

2. RuleBasedEngine

Ο πυρήνας του συστήματος είναι ένα δυναμικό SPARQL query που εκτελείται στον Apache Jena Fuseki. Το σύστημα εντοπίζει και μετρά σημασιολογικές σχέσεις διασχίζοντας την οντολογία μέσω συγκεκριμένων σημασιολογικών διαδρομών (semantic paths). Το SPARQL query υπολογίζει πόσες φορές ικανοποιείται ένα κριτήριο, δηλαδή πόσες σχετικές σημασιολογικές διαδρομές εντοπίζονται για κάθε Training Centre. Τα αποτελέσματα αυτής της διαδικασίας αποτελούν τη βάση για τον υπολογισμό των επιμέρους Scores.

Τα κριτήρια μπορούν να χωριστούν σε δύο κατηγορίες, τα δυναμικά και τα στατικά κριτήρια.

2.1. Dynamic Criteria (Εξαρτώνται από το Input)

Αυτά τα κριτήρια καθορίζουν τη σχετικότητα και υπολογίζονται δυναμικά με βάση τις επιλογές του χρήστη.

1. **Tech Use:** Διαθέτει το Κέντρο την επιλεγμένη τεχνολογία;
 - *Semantic Path:* TC --[usesTechnology]--> Selected_Technology
2. **Tech Training:** Παρέχει το Κέντρο μάθημα που εκπαιδεύει πάνω στη συγκεκριμένη τεχνολογία;
 - *Semantic Path:* TC --[providesCourse]--> Course --[trainsOn]--> Selected_Technology
3. **Incident Coverage:** Έχει το Κέντρο εμπειρία σε περιστατικά που αποτελούν τη βάση του επιλεγμένου σεναρίου;

- *Semantic Path*:TC --[tacklesIncident]--> Incident <--[isBasedOnIncident]-- Selected_Scenario

4. **Threat Capability**: Διαθέτει το Κέντρο πόρους για την απειλή που εμπλέκεται στο σενάριο;

- *Semantic Path*:TC --[usesTech]-->Selected_Technology--[addressesThreat]--> Threat <--[involvesThreat]-- Selected_Scenario

2.2. Static Criteria (Γενικές Δυνατότητες)

Αυτά τα κριτήρια είναι ανεξάρτητα από το input και μετρούν τη συνολική δυναμικότητα και τις υποδομές του κέντρου.

5. **Facilities**: Πλήθος εγκαταστάσεων.

- *Semantic Path*:TC --[hasFacility]--> Facility

6. **Disciplines**:Πλήθοςεπιστημονικώνκλάδων (Displines).

- *Semantic Path*:TC --[hasTCDiscipline]--> Discipline

7. **Courses**: Συνολικό πλήθος μαθημάτων που προσφέρει το κέντρο.

- *Semantic Path*:TC --[providesTrainingCourse]--> Course

8. **Networks**: Πλήθος δικτύων στα οποία συμμετέχει.

- *Semantic Path*:TC --[connectsWithNetwork]--> Network

Κάθε κριτήριο μπορεί να λάβει τιμές σε διαφορετικό εύρος, ανάλογα με τη φύση του. Ορισμένα κριτήρια είναι δυαδικά, λαμβάνοντας τιμές από **0 έως 1**, ενώ άλλα βασίζονται σε πλήθος εμφανίσεων semantic paths και μπορούν να λάβουν τιμές έως και **0–33**, ανάλογα με το πόσες σχετικές τριπλέτες εντοπίζονται στο γραφικό μοντέλο.

Για παράδειγμα, το κριτήριο **Tech Use** είναι δυαδικό. Λαμβάνει τιμή **1** όταν το SPARQL query εντοπίζει semantic path της μορφής:

TC —[usesTechnology]→ Selected_Technology

δηλαδή όταν το Κέντρο πράγματι χρησιμοποιεί την επιλεγμένη τεχνολογία. Αν δεν εντοπιστεί αντίστοιχη διαδρομή, το κριτήριο λαμβάνει τιμή **0**.

Αντίθετα, κριτήρια όπως το **Networks** βασίζονται σε πολλαπλές εμφανίσεις semantic paths. Το σύστημα εντοπίζει όλες τις διαδρομές της μορφής:

TC —[connectsWithNetwork]→ Network

και ο αριθμός των δικτύων που βρίσκει μετατρέπεται στο αντίστοιχο αριθμητικό αποτέλεσμα του κριτηρίου. Έτσι, το τελικό score του κριτηρίου αντιπροσωπεύει το πλήθος των δικτύων στα οποία συμμετέχει το Κέντρο.

3. Ο Αλγόριθμος Βαθμολόγησης (Scoring Algorithm)

Η κατάταξη των κέντρων ακολουθεί τις αρχές της Πολυκριτηριακής Λήψης Αποφάσεων (MCDM), χρησιμοποιώντας ιεραρχική δομή (Clustering).

3.1. Κανονικοποίηση (Normalization)

Για να συγκριθούν ανόμοια μεγέθη, εφαρμόζεται κανονικοποίηση βάσει της μέγιστης τιμής που παρατηρείται στα τρέχοντα αποτελέσματα.

$$\bullet \text{ Τύπος: } V_{norm-i} = \frac{V_i}{V_{max}}$$

Όπου V_{norm-i} : η κανονικοποιημένη τιμή, V_i : η πραγματική τιμή που εμφανίζει το κριτήριο, V_{max} : η μέγιστη τιμή που εμφανίζει το κριτήριο σε κάποιο κέντρο με δεδομένη τεχνολογία και σενάριο.

3.2. Clustering&Weights

Τα κριτήρια ομαδοποιούνται σε Clusters. Τα βάρη έχουν επιλεγεί με βάση την κρισιμότητα, όπως ορίζεται σε πρότυπα NATO και ISO και τον βαθμό συσχέτισης με το ζητούμενο.

A. Operational Fit (Cluster Σχετικότητας)

Εξαρτάται από τα Dynamic Criteria.

$$\text{Τύπος: } Score_{op} = 0.35 \cdot TechUse + 0.20 \cdot TechTraining + 0.25 \cdot IncidentCoverage + 0.20 \cdot ThreatCapability$$

Τεκμηρίωση Βαρών:

1. Tech Use (0.35) & Training (0.20): Η διαθεσιμότητα φυσικών πόρων και εξοπλισμού θεωρείται κρίσιμη παράμετρος για την ανθεκτικότητα (Resilience) κατά το πρότυπο **ISO 22300**. Η έλλειψη ειδικού εξοπλισμού (π.χ. Mass Spectrometer) καθιστά την εκπαίδευση αδύνατη (hard constraint), γι' αυτό λαμβάνει το υψηλότερο βάρος.

2. Threat Capability & Incident: Σύμφωνα με τα πρότυπα **NATO ATP-3.8.1 (CBRN Defence Standards)**, η ικανότητα εξουδετέρωσης συγκεκριμένων παραγόντων (Threat Agents) είναι ο πρωταρχικός δείκτης επιτυχίας μιας μονάδας. Τα κριτήρια αυτά λειτουργούν συμπληρωματικά για να πλαισιώσουν την τεχνολογία στο επιχειρησιακό περιβάλλον.

B. Training Capacity (Cluster Δυναμικότητας)

- Τύπος: $Score_{cap} = 0.60 \cdot Courses + 0.40 \cdot Disciplines$

C. Infrastructure & Networks (Cluster Υποστήριξης)

- Τύπος: $Score_{infra} = 0.60 \cdot Facilities + 0.40 \cdot Networks$

3.3. Υπολογισμός Τελικού Σκορ & Core Score Logic

1. Base Score:

$$BaseScore = 0.65 \cdot Score_{op} + 0.20 \cdot Score_{infra} + 0.15 \cdot Score_{cap}$$

Το Operational Fit κυριαρχεί (65%), διασφαλίζοντας ότι προτείνονται κέντρα που είναι σχετικά και όχι απλώς "μεγάλα".

Σημείωση: Σε μελλοντική έκδοση της εφαρμογής, το σύστημα δύναται να ενσωματώσει λειτουργικότητα εξατομίκευσης βαρών. Μέσω αυτής, ο χρήστης θα έχει τη δυνατότητα να ρυθμίζει δυναμικά τις προτεραιότητες του αλγορίθμου, επιλέγοντας αν η σύσταση θα εστιάζει πρωτίστως στην Καταλληλότητα (Operational Fit) ή στη Συνολική Δυναμικότητα του κέντρου. Επιπλέον, θα μπορούσε να παρέχεται η ευελιξία καθορισμού της σχετικής βαρύτητας μεταξύ Τεχνολογίας και Σεναρίου, επιτρέποντας την προσαρμογή του μοντέλου σε ειδικές απαιτήσεις (Technology-centric vs Scenario-centric approach).

4. UserInterface, Explainability&Feedback

4.1. ΔυναμικήΦόρτωση&Φιλτράρισμα

Το Frontend αντλεί τις λίστες επιλογών δυναμικά (/api/options). Χρησιμοποιείται φίλτρο owl:NamedIndividual στο SPARQL ώστε τα dropdowns να περιέχουν μόνο συγκεκριμένα instances (π.χ. "Foam disinfectants") και όχι αφηρημένες κλάσεις, διασφαλίζοντας την ακρίβεια των queries.

4.2. Explainability (XAI)

Το σύστημα ενισχύει την εμπιστοσύνη μέσω δύο επιπέδων επεξήγησης:

- **Simple Explanations:** Κείμενο φυσικής γλώσσας που εξηγεί γιατί επιλέχθηκε ένα κέντρο.
- **Justification Graph:** Οπτικοποίηση των ακριβών τριπλετών RDF που συνδέουν το Κέντρο με το Αίτημα (π.χ. Center A → providesCourse → Course B). Αυτό προσφέρει διαφάνεια.

4.3. Μηχανισμός Feedback

Το σύστημα καταγράφει τις αντιδράσεις των χρηστών (Bad/Neutral/Good) σε αρχείο JSONL. Σε περιπτώσεις πολλαπλών αλληλεπιδράσεων από τον ίδιο χρήστη για το ίδιο αποτέλεσμα ("αναποφάσιστος χρήστης"), για την ανάλυση πρέπει να λαμβάνεται υπόψη η εγγραφή με το μεταγενέστερο timestamp, καθώς αντικατοπτρίζει την τελική κρίση του χρήστη.

5. Διάγραμμα Ροής Συστήματος

Σύνοψη Διαδικασίας:

1. Το **Frontend** ζητά Σύσταση από το Backend.
2. Το **Backend** κατασκευάζει το ερώτημα SPARQL.
3. Ο **Fuseki Server** εκτελεί το ερώτημα στην Οντολογία.
4. Το **Backend** επεξεργάζεται τα αποτελέσματα, υπολογίζει μαθηματικά σκορ (χρησιμοποιώντας βάρη) και παράγει επεξηγήσεις.
5. Το **Frontend** αποδίδει την καταταγμένη λίστα με τα μονοπάτια αιτιολόγησης.

6. WP0 – Synthetic Ground Truth & Offline Evaluation

6.1 Σκοπός του WP0

Σκοπός του WP0 είναι η δημιουργία ενός επιστημονικά έγκυρου και επαναλήψιμου πλαισίου αξιολόγησης (offline evaluation) για το προτεινόμενο σύστημα συστάσεων, σε ένα περιβάλλον όπου δεν υπάρχουν διαθέσιμα πραγματικά δεδομένα χρηστών (user interaction logs, ratings, clicks).

6.2 Γιατί Synthetic Ground Truth

Ελλείψει πραγματικών χρηστών, το σύστημα αξιολογείται μέσω ενός idealized expert oracle, ο οποίος προσομοιώνει τη λογική ενός έμπειρου αξιολογητή CBRN:

Το oracle δεν αποτελεί «πραγματική αλήθεια», αλλά μια ελεγχόμενη και τεκμηριωμένη προσέγγιση relevance judgement.

Η επιλογή αυτή επιτρέπει σύγκριση διαφορετικών μεθόδων (baseline, WP1, WP2, WP3) και καθιστά δυνατή την επαναληψιμότητα των πειραμάτων.

6.3 Δομή του Dataset (WP0)

Για το WP0 δημιουργείται ένα dataset ερωτημάτων τύπου Q1:

Q1: (Technology, Scenario) → Training Centres

Κάθε εγγραφή του dataset (**data_wp0_q1.jsonl**) περιλαμβάνει:

- qid: μοναδικό αναγνωριστικό ερωτήματος
- qtype: τύπος ερωτήματος (π.χ. Q1)
- query: τα IRIs της επιλεγμένης τεχνολογίας και του σεναρίου
- candidates: λίστα υποψήφιων Training Centres
- labels_1_5: βαθμολογία relevance σε κλίμακα 1–5
- labels_bin: δυαδική relevance (relevant / non-relevant)
- oracle_meta: βοηθητικά στοιχεία (utilities, ενεργοποιημένοι κανόνες) για ανάλυση και explainability

6.4 Δημιουργία των Relevance Labels

6.4.1 Utility-based αξιολόγηση

Ο oracle υπολογίζει ένα **utility score** για κάθε υποψήφιο κέντρο, βασισμένο σε σημασιολογικά σήματα (signals) που προκύπτουν από την οντολογία, όπως:

- χρήση ή εκπαίδευση σε συγκεκριμένη τεχνολογία,
- κάλυψη σχετικών περιστατικών (incidents)
- ικανότητα αντιμετώπισης απειλών (μέσω en:addressesThreat),
- υποδομές και δίκτυα.

Τα σήματα αυτά αντιστοιχούν σε **semantic paths** στο knowledge graph και αποτελούν μετρήσιμη ένδειξη καταλληλότητας.

Ουσιαστικά το wr0 ταυτίζεται με το wr1 απλά είναι offline.

6.4.2 Μετατροπή σε Labels

Από το utility score παράγονται:

- **ordinal labels (1–5)** για graded relevance (χρήσιμα για nDCG),
- **binary labels (labels_bin)**, όπου ως relevant ορίζονται τα top-N κέντρα (στην παρούσα εργασία N=2).

Η επιλογή του top-2 αποτελεί **σχεδιαστική παραδοχή** του WP0 και δηλώνεται ρητά, ώστε να μην γενικεύεται πέραν του πλαισίου αξιολόγησης.

6.5 Διαχωρισμός Δεδομένων (Train / Validation / Test)

Ο διαχωρισμός των δεδομένων (**data_wp0_splits.json**) γίνεται ανά query id (qid) και όχι ανά candidate, ώστε να αποφεύγεται leakage.

- Train set: ρύθμιση παραμέτρων / πειραματισμός
- Validation set: επιλογή παραμέτρων
- Test set: τελική αναφορά αποτελεσμάτων

Η διαδικασία είναι αναπαραγώγιμη μέσω σταθερού seed.

6.6 Μετρικές Αξιολόγησης

Η αξιολόγηση γίνεται με καθιερωμένες μετρικές ranking:

- **Precision@k**: ποσοστό relevant αποτελεσμάτων στα top-k
- **Recall@k**: ποσοστό relevant που ανακτήθηκαν
- **HitRate@k**: αν υπάρχει τουλάχιστον ένα relevant στα top-k
- **nDCG@k**: graded ranking quality (κύρια μετρική)
- **MRR**: θέση πρώτου relevant αποτελέσματος

Οι μετρικές αυτές είναι τυπικές στη βιβλιογραφία recommender systems και επιτρέπουν άμεση σύγκριση διαφορετικών μεθόδων.

6.7 Τι επιτυγχάνει το WPO

Το WPO:

- καθιερώνει ένα **σταθερό και συγκρίσιμο evaluation framework**,
- επιτρέπει αντικειμενική σύγκριση μεταξύ διαφορετικών προσεγγίσεων,
- λειτουργεί ως βάση για όλα τα επόμενα Work Packages.

6.8 Δομή Φάκελου recsys_wp0 (ενδέχεται να υπάρχουν μικρές διαφορές με την τρέχουσα υλοποίηση)

recsys_wp0/__init__.py

Δηλώνει ότι το WPO είναι Python package, επιτρέπει imports τύπου “from recsys_wp0”

recsys_wp0/kg_loader.py

Φορτώνει το Knowledge Graph (ontology) σε Python, για offline processing.

- Διαβάζει το ontology.ttl σε rdflib Graph.
- Επιστρέφει rdflib Graph έτοιμο για queries/triple traversal..

recsys_wp0/query_space.py

- Εντοπίζει διαθέσιμες **technologies, scenarios, centres** από το KG (με SPARQL-like rdflib queries / triple patterns).
- Παρέχει helpers για να:
 - φτιάξουμε Q1 queries (technology+scenario),
 - επιλέξουμε υποψήφια centres (σταθερά 5 candidates ανά query στο WPO).

recsys_wp0/q1_features.py

Είναι το “feature extractor” για Q1:

Υπολογίζει **semantic signals** (μετρικές/ενδείξεις) από paths της οντολογίας.

Το κεντρικό API:

- `compute_q1_signals(g, tech_iri, scen_iri, centre_iri) -> Dict[str, float]`

Επιστρέφει dict με signals (ενδεικτικά):

- `tech_use`: το κέντρο **χρησιμοποιεί** την τεχνολογία
- `tech_train`: το κέντρο **εκπαιδεύει** σε τεχνολογία μέσω course path (π.χ. *Centre* → *providesCourse* → *Course* → *trainsOn* → *Technology*)
- `incident`: overlap/συσχέτιση με incidents του scenario
- `threat_cap`: ικανότητα αντιμετώπισης απειλών μέσω `addressesThreat`
- `facility`: υποδομές/εγκαταστάσεις
- `discipline`: B/C/RN κάλυψη
- `course`: διαθέσιμα courses/training profile
- `network`: συμμετοχή σε networks (connectivity)

recsys_wp0/oracle_q1.py

Ο “Oracle” παράγει **ground truth** labels από τα signals.

- `compute_utility(signals, weights, noise_sigma, seed)`
Υπολογίζει utility score ανά centre ως weighted sum των signals + προαιρετικό noise (για ρεαλισμό).
- `labels_from_utilities(utilities)`
Παράγει:
 - `labels_1_5` (rank-based 1..5)
 - `labels_thresh_0_5` (threshold-based 0..5 από utility cutoffs)
 - `labels_bin` (binary relevance, π.χ. top-N centres)

Επίσης κρατάει `oracle_meta`:

- `utilities, rules_fired, noise params, seed`

Γιατί υπάρχει:

Χωρίς πραγματικά user logs, χρειαζόμαστε “ideal expert” proxy για να μετρήσουμε αλγορίθμους offline.

recsys_wp0/baseline_model.py

Baseline ranker για Q1.

Κεντρική συνάρτηση:

- `baseline_rank_from_signals(signals_by_center) -> List[(centre_iri, score)]`

Κάνει:

- weighted sum / normalization,
- επιστρέφει ranked list.

Γιατί υπάρχει:

Χρειαζόμαστε baseline για σύγκριση και sanity checks (να δούμε ότι τα signals “βγάζουν” λογικά rankings).

recsys_wp0/splitter.py

Δημιουργεί train/val/test split σε επίπεδο query (qid).

- Σταθερό seed για αναπαραγωγιμότητα.
- Επιστρέφει dict με λίστες από qids.

Γιατί υπάρχει:

Ο διαχωρισμός πρέπει να είναι σταθερός, ώστε όλα τα επόμενα WPs να συγκρίνονται δίκαια.

recsys_wp0/dataset_wp0_q1.py

Το βασικό script παραγωγής dataset για WP0.

Τι κάνει end-to-end:

1. Φορτώνει KG (kg_loader.py)
2. Δημιουργεί Q1 queries + candidates (query_space.py)
3. Υπολογίζει signals ανά centre (q1_features.py)
4. Υπολογίζει utilities + labels μέσω Oracle (oracle_q1.py)
5. Γράφει data_wp0_q1.jsonl και data_wp0_splits.json

Γιατί υπάρχει:

Είναι ο “dataset generator” του WP0 και αποτελεί το entry point για να ξαναχτίσουμε το benchmark όποτε αλλάξει η οντολογία.

recsys_wp0/eval_metrics.py

Υλοποιεί standard ranking metrics για recommender evaluation.

- precision_at_k, recall_at_k, hitrate_at_k
- ndcg_at_k, mrr
- Τα metrics δουλεύουν πάνω σε:
 - y_true_all: relevance dicts (π.χ. labels_bin ή graded relevance)
 - y_score_all: score dicts από τον αλγόριθμο.

recsys_wp0/eval_runner.py

Runner αξιολόγησης baseline πάνω στο WP0 dataset.

- Διαβάζει data_wp0_q1.jsonl + data_wp0_splits.json
- Για κάθε query:
 - υπολογίζει baseline ranking/score
 - συγκρίνει με labels (συνήθως labels_bin για P@k/R@k/HR@k, και graded για nDCG)
- Επιστρέφει JSON report με metrics ανά split.

recsys_wp0/wp0_rebuild_labels_bin.py

Utility script για επανυπολογισμό του labels_bin χωρίς να ξαναχτίσουμε όλο το dataset.

- Διαβάζει υπάρχον data_wp0_q1.jsonl
- Ξαναγράφει labels_bin ως top-N από labels_1_5 ή utilities (ανάλογα με την υλοποίηση)
- Χρήσιμο όταν αλλάξουμε ορισμό relevance (π.χ. top-2 → top-3) και θέλουμε γρήγορη ενημέρωση.

Γιατί υπάρχει:

Μικρή αλλαγή definition → δεν χρειάζεται full regeneration του dataset.

6.9 Πώς “δένουν” όλα μεταξύ τους (μία γραμμική περιγραφή)

1. **Ontology → Graph**
kg_loader.py φορτώνει ontology.ttl.
2. **Query generation**
query_space.py επιλέγει έγκυρα (technology, scenario) και φτιάχνει 5 candidate centres.
3. **Signal extraction**
q1_features.py μετράει semantic paths/signals ανά (centre, tech, scenario).
4. **Oracle ground truth**
oracle_q1.py μετατρέπει τα signals σε utility → labels (labels_1_5, labels_thresh_0_5, labels_bin) και αποθηκεύει explainable metadata (oracle_meta).
5. **Dataset + splits**
dataset_wp0_q1.py γράφει data_wp0_q1.jsonl και splitter.py γράφει data_wp0_splits.json.
6. **Baseline evaluation**
baseline_model.py παράγει scores και eval_runner.py + eval_metrics.py βγάζουν offline metrics.

7. WP1 – Meta-path Evidence Layer (SPARQL-based)

7.1 Σκοπός του WP1

Σκοπός του WP1 είναι η τυπική διατύπωση και απομόνωση της path-based λογικής του συστήματος συστάσεων, μέσω ρητά ορισμένων meta-paths πάνω στο knowledge graph. **(Είναι ακριβώς η ίδια λογική με το app, δεν αλλάζει κάτι ουσιαστικά, ο σκοπός του είναι για tests/evaluation κλπ.)**

Το WP1 λειτουργεί ως:

- Καθαρό **symbolic / structural recommender layer**,
- ανεξάρτητο baseline,
- Σημείο αναφοράς για πιο σύνθετες υβριδικές προσεγγίσεις.

7.2 Γιατί WP1 ενώ υπάρχει ήδη recommender

Ο κύριος recommender του συστήματος είναι σχεδιασμένος με γνώμονα τη λειτουργικότητα και την εμπειρία χρήστη.

Το WP1, αντίθετα, εξυπηρετεί ακαδημαϊκό και πειραματικό σκοπό:

- απομονώνει τη σημασιολογική δομή (paths),
- επιτρέπει ablation studies,
- παρέχει καθαρή σύγκριση με WP2 (MCDM) και WP3 (embeddings).

7.3 Ορισμός Meta-paths

Κάθε meta-path ορίζεται ρητά ως ακολουθία σχέσεων στην οντολογία. Ενδεικτικά:

- **MP1 – Technology Usage**
TrainingCentre → usesTechnology → Technology
- **MP2 – Incident Coverage**
TrainingCentre → tacklesIncident → Incident → partOfScenario → Scenario
- **MP3 – Threat Capability**
TrainingCentre → hasResource → Resource → addressesThreat → Threat

Κάθε meta-path αντιπροσωπεύει συγκεκριμένη επιχειρησιακή έννοια.

7.4 Structural Scoring

Για κάθε query, τα meta-paths εκτελούνται μέσω SPARQL στο Fuseki endpoint.

Τα αποτελέσματα μετατρέπονται σε numeric signals (counts), τα οποία κανονικοποιούνται, συνδυάζονται σε structural score ανά κέντρο.

Το αποτέλεσμα είναι μια καθαρά path-based κατάταξη.

7.5 Αξιολόγηση WP1

Το WP1 αξιολογείται **στο ίδιο dataset και με τις ίδιες μετρικές του WP0**, εξασφαλίζοντας πλήρη συγκρισιμότητα.

Η χρήση της ίδιας διαδικασίας αξιολόγησης επιτρέπει άμεση σύγκριση baseline vs WP1, έλεγχο της συνεισφοράς των meta-paths, τεκμηρίωση των επόμενων βημάτων.

7.6 Δομή φακέλου recsys_wp1(ενδέχεται να υπάρχουν μικρές διαφορές με την τρέχουσα υλοποίηση)

recsys_wp1/__init__.py

Δηλώνει το WP1 ως Python package

recsys_wp1/sparql_client.py

Wrapper για επικοινωνία με Apache Fuseki SPARQL endpoint.

Τι κάνει:

- ορίζει prefixes,
- στέλνει SPARQL queries στο endpoint,
- επιστρέφει αποτελέσματα σε Python-friendly μορφή.

Γιατί υπάρχει:

Το WP1 λειτουργεί online πάνω στον KG, ώστε:

- να μην χαθεί η δουλειά που έχει γίνει με SPARQL,
- να χρησιμοποιείται ακριβώς η ίδια λογική με την εφαρμογή.

recsys_wp1/metapaths.py

Ορίζει ρητά όλα τα meta-paths (MP1–MP8) που χρησιμοποιούνται.

Κάθε meta-path περιγράφεται από:

- mp_id (π.χ. "MP1")
- name
- description
- count_fn: συνάρτηση που υπολογίζει το feature
- evidence_fn: συνάρτηση που συλλέγει explainability evidence

Ενδεικτικά meta-paths:

- **MP1 — Direct Technology Capability**
Centre → usesTechnology → Technology
- **MP2 — Technology Training**
Centre → providesCourse → Course → trainsOn → Technology
- **MP3 — Incident Coverage**
- **MP4 — Threat Coverage (addressesThreat)**
- **MP5 — Infrastructure / Facilities**
- **MP6 — Discipline Coverage (B/C/RN)**

- **MP7 — Network Participation**
- **MP8 — Experience / Courses**

Σημαντικό:

Τα meta-paths του WP1 είναι **πλήρως ευθυγραμμισμένα** με:

- τα signals του q1_features.py,
- τα clusters/κριτήρια του recommender.py.

recsys_wp1/features_wp1.py

Υλοποιεί την εξαγωγή meta-path features μέσω SPARQL.

Τι κάνει:

- για κάθε (technology, scenario, centre):
 - εκτελεί SPARQL queries για κάθε meta-path,
 - επιστρέφει:
 - raw counts (π.χ. πόσες courses, πόσα networks),
 - evidence (URIs που ενεργοποίησαν το path).

recsys_wp1/structural_ranker_wp1.py

Ο πυρήνας του WP1 recommender.

Τι κάνει βήμα-βήμα:

1. Παίρνει ως είσοδο:
 - technology_iri
 - scenario_iri
 - λίστα centres
2. Καλεί features_wp1.py για να πάρει meta-path features.
3. Κανονικοποιεί τα features (min-max).
4. Υπολογίζει **structural score**:
5. $score_c = \sum (weight_i \cdot feature_i)$
6. Επιστρέφει:
 - ranked list centres,
 - scores,
 - evidence (αν ζητηθεί).

recsys_wp1/wp1_eval_runner.py

Runner αξιολόγησης του WP1 πάνω στο **ίδιο dataset του WP0**.

Τι κάνει:

- διαβάζει data_wp0_q1.jsonl,
- για κάθε query:

- ο τρέχει το WP1 ranker,
 - ο συγκρίνει το ranking με τα labels_bin / labels_1_5,
- υπολογίζει metrics μέσω recsys_wp0.eval_metrics.

Γιατί είναι σημαντικό:

Επιτρέπει άμεση, δίκαιη σύγκριση:

- WP0 baseline vs WP1 structural,
- χωρίς αλλαγή dataset ή split.

8 WP2 — Multi-Criteria Decision Making (MCDM) με Entropy Weights & TOPSIS

8.1 Σκοπός του WP2

Το WP2 εισάγει ένα **τυπικό Multi-Criteria Decision Making (MCDM) framework**, με στόχο:

- να μετατρέψει τα **δομικά meta-path features του WP1** σε **κριτήρια απόφασης**,
- να παρέχει **θεωρητικά θεμελιωμένη κατάταξη** Training Centres,
- να μειώσει την αυθαιρεσία των απλών weighted sums,

Σε αντίθεση με το WP1, όπου το score υπολογίζεται ως:

$$score_c = \sum (weight_i \cdot feature_i)$$

στο WP2 η απόφαση προκύπτει μέσω **TOPSIS** (απόστασης κάθε κέντρου από την ιδανική και την αντι-ιδανική λύση)

8.2 Τι είναι το MCDM

Το Multi-Criteria Decision Making αφορά προβλήματα όπου:

- υπάρχουν **πολλαπλά κριτήρια**,
- τα κριτήρια είναι **ετερογενή**,
- δεν υπάρχει μία προφανής “σωστή” λύση.

Στο πρόβλημά μας:

- Εναλλακτικές (alternatives): Training Centres
- Κριτήρια (criteria): Meta-paths MP1–MP8

- Στόχος: Κατάταξη κέντρων για συγκεκριμένο (technology, scenario)

8.3 Entropy Weighting — Υπολογισμός Βαρών από τα Δεδομένα

8.3.1 Στο WP1 τα weights είναι δοσμένα από τον domain σχεδιασμό (π.χ. tech use > networks).

Στο WP2 δεν τα αντικαθιστούμε, αλλά:

- χρησιμοποιούμε Entropy Weights για να εκτιμήσουμε πόση πληροφορία φέρει κάθε κριτήριο,
- επιτρέπουμε data-driven διαφοροποίηση μεταξύ queries,
- διατηρούμε επιστημονική αυστηρότητα.

8.3.2 Μαθηματικός Ορισμός

Έστω:

- m : αριθμός εναλλακτικών (centres)
- n : αριθμός κριτηρίων (meta-paths)
- x_{ij} : τιμή κριτηρίου j για κέντρο i

Βήμα 1 - Κανονικοποίηση

$$p_{ij} = \frac{x_{ij}}{\sum_{i=1}^m x_{ij}}$$

Βήμα 2 - Entropy

$$e_j = -k \sum_{i=1}^m p_{ij} \cdot \ln(p_{ij}), k = \frac{1}{\ln(m)}$$

Βήμα 3 - Degree of Divergence

$$d_j = 1 - e_j$$

Βήμα 4 - Entropy Weights

$$w_j^{(entropy)} = \frac{d_j}{\sum_{j=1}^n d_j}$$

8.3.3 Hybrid Weighting Strategy

Στην παρούσα εργασία δεν αντικαθιστούμε τα υπάρχοντα weights. Χρησιμοποιούμε hybrid weighting:

$$w_j = a \cdot w_j^{(manual)} + (1 - a) \cdot w_j^{(entropy)}$$

Όπου:

- $w_j^{(manual)}$: weights από recommender / domain logic
- $w_j^{(entropy)}$: data – driven weights
- α : βάρος

8.4 TOPSIS — Technique for Order Preference by Similarity to Ideal Solution

8.4.1 Βασική Ιδέα

Η καλύτερη λύση είναι αυτή που:

- είναι **πιο κοντά** στην ιδανική λύση,
- και **πιο μακριά** από την αντι-ιδανική.

8.4.2 Μαθηματική Διατύπωση TOPSIS

Βήμα 1 - Decision Matrix

$$X = [x_{ij}], \quad i = 1, \dots, m, j = 1, \dots, n$$

Βήμα 2 - Κανονικοποίηση

$$r_{ij} = \frac{x_{ij}}{\sum_{i=1}^m x_{ij}}$$

Βήμα 3 - Weighted Normalized Matrix

$$v_{ij} = w_j \cdot r_{ij}$$

Βήμα 4 - Ideal / Anti-Ideal Solutions

$$A^+ = \{\max_i v_{ij}\}, \quad A^- = \{\min_i v_{ij}\}$$

Βήμα 5 – Distances

$$D_i^+ = \sqrt{\sum_j (v_{ij} - A_j^+)^2}, \quad D_i^- = \sqrt{\sum_j (v_{ij} - A_j^-)^2}$$

Βήμα 6 - Closeness Coefficient

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}$$

Όσο μεγαλύτερο το C_i , τόσο καλύτερη η λύση.

8.5 Δομή Φακέλου recsys_wp2(ενδέχεται να υπάρχουν μικρές διαφορές με την τρέχουσα υλοποίηση)

recsys_wp2/__init__.py

Δηλώνει το WP2 ως **MCDM decision layer**.

recsys_wp2/ranker_wp2.py

Ο πυρήνας του WP2.

Τι κάνει:

1. Καλεί το WP1 για meta-path features.
2. Κανονικοποιεί features.
3. Υπολογίζει entropy weights.
4. Συνδυάζει με manual weights.
5. Εκτελεί TOPSIS.
6. Παράγει:
 - ranking centres,
 - TOPSIS scores,
 - optional evidence.

recsys_wp2/wp2_eval_runner.py

Αξιολογεί το WP2 πάνω στο ίδιο dataset του WP0.

- ίδια splits,
- ίδια metrics,
- άμεση σύγκριση WP1 vs WP2.

9 WP3 — Knowledge Graph Embeddings (TransE)

9.1 Σκοπός του WP3

Το WP3 δεν αντικαθιστά τα προηγούμενα στάδια, αλλά λειτουργεί ως additive layer.

9.2 Knowledge Graph Embeddings

Ένα Knowledge Graph αποτελείται από τριπλέτες της μορφής (h, r, t)

όπου:

- h : head entity (π.χ. TrainingCentre),
- r : relation (π.χ. usesTechnology),
- t : tail entity (π.χ. Technology).

Τα KGE μοντέλα μαθαίνουν έναν χαμηλοδιάστατο διανυσματικό χώρο, όπου κάθε οντότητα και κάθε σχέση αναπαρίσταται ως διάνυσμα. Στόχος είναι, μέσα σε αυτόν τον χώρο, οι έγκυρες τριπλέτες να έχουν μικρή "απόσταση" μεταξύ τους.

9.2 Το TransE Μοντέλο

Pseudocode:

Algorithm 1 Learning TransE

input Training set $S = \{(h, \ell, t)\}$, entities and rel. sets E and L , margin γ , embeddings dim. k .

- 1: **initialize** $\ell \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each $\ell \in L$
- 2: $\ell \leftarrow \ell / \|\ell\|$ for each $\ell \in L$
- 3: $e \leftarrow \text{uniform}(-\frac{6}{\sqrt{k}}, \frac{6}{\sqrt{k}})$ for each entity $e \in E$
- 4: **loop**
- 5: $e \leftarrow e / \|e\|$ for each entity $e \in E$
- 6: $S_{batch} \leftarrow \text{sample}(S, b)$ // sample a minibatch of size b
- 7: $T_{batch} \leftarrow \emptyset$ // initialize the set of pairs of triplets
- 8: **for** $(h, \ell, t) \in S_{batch}$ **do**
- 9: $(h', \ell, t') \leftarrow \text{sample}(S'_{(h, \ell, t)})$ // sample a corrupted triplet
- 10: $T_{batch} \leftarrow T_{batch} \cup \{((h, \ell, t), (h', \ell, t'))\}$
- 11: **end for**
- 12: Update embeddings w.r.t.
$$\sum_{((h, \ell, t), (h', \ell, t')) \in T_{batch}} \nabla [\gamma + d(h + \ell, t) - d(h' + \ell, t')]_+$$
- 13: **end loop**

Ο μηχανισμός εκπαίδευσης του μοντέλου TransE, υλοποιήθηκε μέσω της βιβλιοθήκης PyKEEN στο αρχείο train_kge.py, τα παραγόμενα διανύσματα χρησιμοποιήθηκαν στο σύστημα συστάσεων (embed_ranker.py).

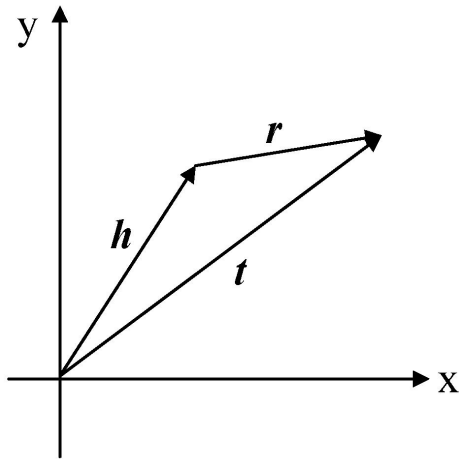
Έστω E το σύνολο όλων των οντοτήτων (Training Centres, Technologies, Scenarios, κ.λπ.) και R το σύνολο των σχέσεων (π.χ. offersTechnologyTraining). Ορίζουμε μια διάσταση ενσωμάτωσης k (embedding dimension), π.χ. $k = 100$.

- Κάθε οντότητα $e \in E$ αντιστοιχίζεται σε ένα διάνυσμα $e \in R^k$
- Κάθε σχέση $r \in R$ αντιστοιχίζεται σε ένα διάνυσμα $r \in R^k$

Κατά την έναρξη της εκπαίδευσης, τα διανύσματα αυτά αρχικοποιούνται με τυχαίες τιμές από μια ομοιόμορφη κατανομή, τοποθετώντας τις οντότητες σε τυχαία σημεία στον k -διάστατο χώρο.

Η θεμελιώδης αρχή του TransE (Bordes et al.) βασίζεται στην εξίσωση:

$$h + r \approx t$$



Δηλαδή, η σχέση r λειτουργεί ως μεταφορά από το head στο tail μέσω του γεωμετρικού χώρου.

9.2.1 Συνάρτηση Ενέργειας

Για κάθε τριπλέτα, υπολογίζουμε την "απόσταση" (dissimilarity score) χρησιμοποιώντας την L1 νόρμα (Manhattan Distance) όπως ορίζεται από την προεπιλογή του PyKEEN:

$$d(\mathbf{h}, \mathbf{r}, \mathbf{t}) = ||\mathbf{h} + \mathbf{r} - \mathbf{t}||$$

Όσο μικρότερο το d , τόσο πιο πιθανή η τριπλέτα. Άρα για την θετική τριπλέτα, στόχος είναι $d_{pos} \rightarrow 0$ ενώ για την αρνητική τριπλέτα $d_{neg} \gg 0$.

9.2.2 Εκπαίδευση του TransE

Η μονάδα εκπαίδευσης είναι η τριπλέτα (h, r, t) . Μια τριπλέτα ονομάζεται:

- **Θετική (Positive):** όταν είναι πραγματική εγγραφή από την οντολογία μας (εξαγόμενη από το Kg_export.py).
- **Αρνητική (Negative):** όταν κατασκευάζεται τεχνητά μέσω της διαδικασίας corruption αντικαθιστώντας είτε το head (h) ή το tail (t) σε μία τυχαία θετική τριπλέτα, αυτή η τριπλέτα θεωρείται ψευδής και το μοντέλο πρέπει να μάθει να την αποφεύγει.

Loss Function (Margin Ranking Loss): Η συνάρτηση απώλειας ορίζεται ως

$$L = \sum \max(0, \gamma + f(\mathbf{h}, \mathbf{r}, \mathbf{t}) - f(\mathbf{h}', \mathbf{r}, \mathbf{t}'))$$

όπου γ είναι το margin, δηλαδή αν $d_{neg} - d_{pos} \geq \gamma$ η απώλεια είναι μηδέν και άρα το μοντέλο έχει μάθει τη συγκεκριμένη σχέση.

Εάν η απώλεια είναι θετική ($L > 0$) εκτελείται το βήμα της βελτιστοποίησης (Stochastic gradient descent). Τα διανύσματα ενημερώνονται ως εξής:

1. Pull: Τα διανύσματα της θετικής τριπλέτας (h, r, t) προσαρμόζονται ώστε το άθροισμα (h, r) να πλησιάζει το t .
2. Push: Τα διανύσματα της αρνητικής τριπλέτας προσαρμόζονται ώστε να απομακρυνθούν από το σημείο πρόβλεψης $h + r$.

Συγκεκριμένα:

Η προσαρμογή γίνεται μέσω της αφαίρεσης της κλίσης (gradient) από το τρέχον διάνυσμα. Ο γενικός κανόνας ενημέρωσης (Update Rule) για οποιαδήποτε παράμετρο θ (είτε είναι h, r, t) είναι:

$$\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$$

Όπου:

- η : Ο Ρυθμός εκμάθησης (Learning Rate). Μια μικρή σταθερά (π.χ. 0.01) που καθορίζει πόσο γρήγορα αλλάζουν τα διανύσματα.
- $\frac{\partial L}{\partial \theta}$: Η Παράγωγος της συνάρτησης απώλειας ως προς το διάνυσμα (η κλίση). Δείχνει προς τα πού μεγαλώνει το σφάλμα.

Στην περίπτωση μας έχουμε:

1. Pull:

- Ενημέρωση head (h): Το h κινείται προς το $t - r$.

$$h \leftarrow h - \eta \cdot \text{sgn}(h + r - t)$$

- Ενημέρωση tail (t): Το t κινείται προς το $h + r$.

$$t \leftarrow t + \eta \cdot \text{sgn}(h + r - t)$$

- Ενημέρωση relation (r): Το r κινείται προς το $h - t$.

$$r \leftarrow r - \eta \cdot \text{sgn}(h + r - t)$$

1. Push Ενημέρωση Αρνητικής Τριπλέτας (h, r, t') , (h', r, t)

Στόχος είναι η αύξηση της απόστασης (μόνο αν αυτή είναι μικρότερη από το margin). Εδώ, κινούμαστε προς την κατεύθυνση που αυξάνει το σφάλμα για την αρνητική τριπλέτα:

- Ενημέρωση Corrupted tail (t'): Το t' απωθείται μακριά από την πρόβλεψη $h + r$.

$$t' \leftarrow t' - \eta \cdot \text{sgn}(h + r - t') \cdot (-1)$$

(Το -1 προκύπτει επειδή στην Loss Function η αρνητική απόσταση αφαιρείται).

Αυτή η διαδικασία επαναλαμβάνεται μέσω της βιβλιοθήκης PyKEEN για πλήθος εποχών (epochs) όπως ορίζεται στο train_kge.py, οδηγώντας στη σταδιακή οργάνωση του χώρου: τα Training Centres που προσφέρουν παρόμοιες τεχνολογίες συγκεντρώνονται σε γειτονικές περιοχές (clustering).

Geometric Constraints (Normalization): Κατά τη διάρκεια της εκπαίδευσης, μετά από κάθε ενημέρωση των βαρών (gradient descent step), εφαρμόζεται ένας περιορισμός κανονικοποίησης ώστε τα διανύσματα των οντοτήτων να έχουν μοναδιαίο μήκος ($\|e\|_2 = 1$). Αυτό αποτρέπει το μοντέλο από το να ελαχιστοποιεί το σφάλμα αυξάνοντας απλώς τα μεγέθη των διανυσμάτων στο άπειρο και διασφαλίζει ότι η σύγκριση μέσω αποστάσεων ή ομοιότητας συνημίτονου (cosine similarity) είναι έγκυρη.

9.2.3 Τι Μαθαίνει Πραγματικά το TransE

Το μοντέλο βλέπει IRIs οντοτήτων, σχέσεις μεταξύ τους τη συνολική δομή γράφου και μέσω εκπαιδεύσεις οντότητες με παρόμοιο ρόλο στον γράφο αποκτούν παρόμοια embeddings και οντότητες με παρόμοιο ρόλο στον γράφο αποκτούν παρόμοια embeddings.

Παρότι το TransE είναι link prediction model, στο WP3 χρησιμοποιείται ως recommender μέσω similarity-based ranking.

Για ένα ερώτημα Q1:

- επιλεγμένη τεχνολογία T ,
- επιλεγμένο σενάριο S ,

ορίζουμε:

$$e_Q = \beta \cdot e_T + (1 - \beta)e_s$$

Αυτό λέγεται **centroid-based query representation** και είναι καθιερωμένη πρακτική σε IR & RS.

Similarity Score

Για κάθε Training Centre c :

$$Score_c^{embed} = \cos(e_Q, e_c)$$

Έτσι το μοντέλο προτείνει κέντρα σημασιολογικά κοντά στο ερώτημα, ακόμη και αν δεν υπάρχουν άμεσα paths.

9.3 Περιορισμοί του TransE και Επιπτώσεις

Παρότι το TransE είναι αποδοτικό, παρουσιάζει συγκεκριμένους περιορισμούς στη μοντελοποίηση πολύπλοκων σχέσεων, οι οποίοι αντιμετωπίζονται από τον υβριδικό σχεδιασμό του WP3:

1. Αδυναμία στις σχέσεις 1-to-N και N-to-N:

Στην οντολογία μας, ένα TrainingCentre (h) μπορεί να προσφέρει πολλές διαφορετικές Technologies (t_1, t_2). Το TransE προσπαθεί να ικανοποιήσει ταυτόχρονα τις εξισώσεις $h + r = t_1$ και $h + r = t_2$. Αυτό οδηγεί μαθηματικά στο συμπέρασμα ότι $t_1 = t_2$.

- *Επίπτωση:* Το μοντέλο τείνει να φέρνει πολύ κοντά τις αναπαραστάσεις διαφορετικών τεχνολογιών που προσφέρονται από το ίδιο κέντρο, μειώνοντας τη διακριτική ικανότητα (resolution).

2. Συμμετρικές Σχέσεις:

Αν υπήρχε σχέση συμμετρίας (π.χ. collaboratesWith), το TransE θα εξανάγκαζε το διάνυσμα της σχέσης r να είναι μηδενικό ($r = 0$) και τα embeddings των δύο οντοτήτων να ταυτίζονται, κάτι που δεν ανταποκρίνεται στην πραγματικότητα.

9.4 Δομή Φακέλου recsys_wp3(ενδέχεται να υπάρχουν μικρές διαφορές με την τρέχουσα υλοποίηση)

- **recsys_wp3/kg_export.py:** Εξάγει την οντολογία σε IRI-IRI triples, φιλτράροντας literals και διατηρώντας μόνο object properties.
- **recsys_wp3/train_kge.py:** Εκπαιδεύει το TransE μέσω PyKEEN και παράγει τα embeddings.
- **recsys_wp3/embed_ranker.py:** Υλοποιεί την κατασκευή του query embedding και το cosine similarity ranking.
- **recsys_wp3/ranker_wp3.py:** Ο υβριδικός ranker που συνδυάζει τα scores του WP2 και WP3 με συντελεστή α .
- **recsys_wp3/wp3_eval_runner.py:** Αξιολογεί το WP3 στο dataset του WP0, επιτρέποντας συγκριτική ανάλυση (embeddings vs structural vs hybrid).

Metrics του wp3 (transE+CosineSimilarity) δηλαδή σύγκριση του wp0 με wp3

"split": "all",

"n_queries": 102,

"metrics": {

"P@1": 0.7647058823529411,

"R@1": 0.38235294117647056,

"HR@1": 0.7647058823529411,

"nDCG@1": 0.7647058823529411,

"P@2": 0.6029411764705882,

"R@2": 0.6029411764705882,

"HR@2": 0.9215686274509803,

"nDCG@2": 0.6395476211888247,

"P@3": 0.49346405228758167,

"R@3": 0.7401960784313726,

"HR@3": 0.9901960784313726,

"nDCG@3": 0.723705079019378,

"P@5": 0.4,

"R@5": 1.0,

"HR@5": 1.0,

"nDCG@5": 0.8535407557042315,

"MRR": 0.8684640522875816

}

}